Computer Organization and Assembly Language

Project Report

TOPIC: IMPLEMENTATION OF CALCULATOR

Course Information

- Course Title: Computer Organization and Assembly Language
- Faculty Name: Dr. Muhammad Asfand-e-yar, Asma Basit
- Semester: Spring 2024

Group Members:

1. Muzammil Ahmed(01-134222-114)
2. Daniyal(01-134221-018)
3. Faisal Ameer(01-134182-093)

CLASS: BS(CS) -3A

Abstract

This report presents the implementation of a simple calculator using assembly language in the Irvine32 environment. The calculator supports basic arithmetic operations: addition, subtraction, multiplication, and division. The project was undertaken by group members Muzammil, Daniyal, and Faisal. This report details the project's objectives, implementation process, code structure, and evaluation of functionality, providing an in-depth examination of how fundamental assembly programming concepts are applied. The experience gained through this project underscores the significance of understanding low-level programming for effective software development.

Introduction

Assembly language programming offers a deeper understanding of how software interacts with hardware. It allows programmers to manipulate hardware directly, leading to optimized and efficient code execution. By working at this low level, developers gain insights into the operation of CPUs, memory management, and other critical components of a computer system. This project aims to consolidate our understanding of computer organization and assembly language by creating a basic calculator. The calculator prompts the user to input two numbers and select an arithmetic operation, then displays the result. This project highlights fundamental assembly programming concepts such as input handling, conditional statements, and basic arithmetic operations, showcasing the intricate details of low-level programming.

Implementing a calculator in assembly language is not only an educational exercise but also a practical one. It helps bridge the gap between high-level programming languages and the underlying hardware. The skills acquired through this project are invaluable for tasks that require direct hardware manipulation or performance optimization, such as embedded systems programming, operating systems development, and performance-critical applications.

Body

Objectives

The main objectives of this project are:

- **Implement a functional calculator in assembly language:** Create a simple calculator capable of performing basic arithmetic operations: addition, subtraction, multiplication, and division. This involves writing assembly code that can handle user inputs, perform calculations, and display results.

- **Understand and apply the principles of arithmetic operations in assembly:** Gain a practical understanding of how arithmetic operations are carried out at the assembly level. This includes learning how the CPU performs addition, subtraction, multiplication, and division, and how these operations are represented in machine code.

- **Develop skills in handling user inputs and outputs using the Irvine32 library:** Learn to manage user interactions, including input reading and result displaying, using the Irvine32 library functions. This includes mastering the use of system calls and interrupts to handle input/output operations efficiently.

These objectives were chosen to provide a comprehensive understanding of assembly language programming and its practical applications. By the end of the project, we expected to have a fully functional calculator and a deeper understanding of the intricacies of assembly language.

Tools and Environment

To develop and test the calculator, the following tools and environment were used:

- **Assembler: MASM (Microsoft Macro Assembler) -** A powerful assembler for x86 microprocessors, used to compile the assembly language code. MASM is widely used for its rich set of features and its ability to produce highly optimized machine code.

- Library: Irvine32 - A collection of procedures for input/output and other functions, specifically designed to work with MASM. The Irvine32 library simplifies many of the tedious aspects of assembly language programming by providing ready-made routines for common tasks such as string handling, input/output operations, and mathematical calculations.

- Development Environment: Windows OS with MASM and Irvine32 library installed, providing a suitable platform for assembly language development and testing. The Windows environment was chosen for its robustness and compatibility with MASM and Irvine32.

The choice of tools and environment was guided by the need for a stable and feature-rich development platform. MASM and Irvine32 together provide a powerful combination that simplifies many aspects of assembly language programming while still allowing for detailed control over the hardware.

Code Structure

The project code is organized into procedures that handle different tasks: displaying the menu, getting user input, performing arithmetic operations, and displaying the result. The modular approach ensures that each functionality is encapsulated within a specific procedure, making the code easier to understand and maintain.

Each procedure is designed to perform a specific task, allowing for a clean and organized codebase. The main procedure coordinates the overall flow of the program, calling appropriate subroutines based on user input. This modular design not only makes the code easier to understand but also simplifies debugging and future enhancements.

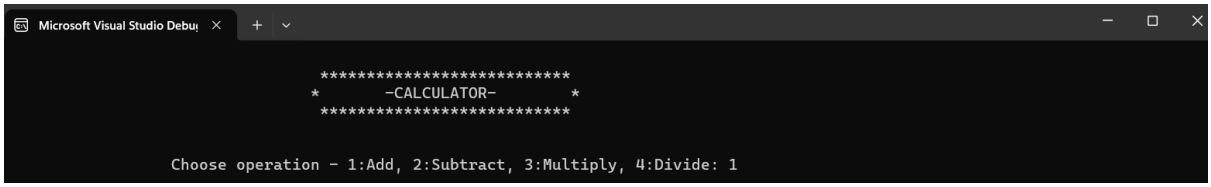
The code structure reflects a well-thought-out design that prioritizes readability and maintainability. By breaking down the program into smaller, manageable pieces, we ensured that each component could be developed and tested independently before being integrated into the main program.

Procedures

The project consists of several procedures, each dedicated to a specific task:

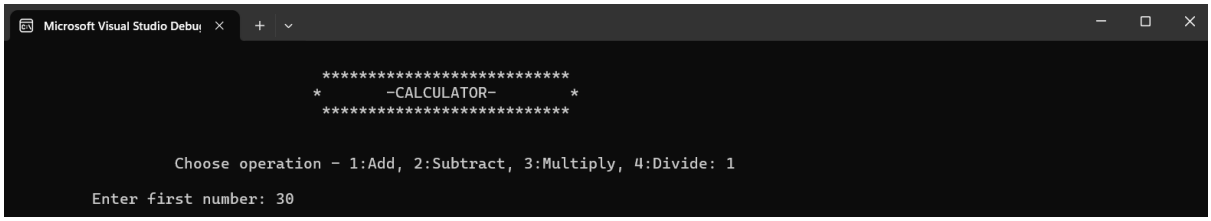
1. **Display Menu:** This procedure displays the calculator title and the menu, prompting the user to choose an arithmetic operation. It provides a user-friendly interface for selecting operations, making the calculator easy to use.

```
displayMenu PROC
    call Crlf
    mov edx, OFFSET Title1
    call WriteString
    call Crlf
    mov edx, OFFSET Title2
    call WriteString
    call Crlf
    mov edx, OFFSET Title3
    call WriteString
    call Crlf
    call Crlf
    call Crlf
    mov edx, OFFSET menu
    call WriteString
    call ReadInt
    ret
displayMenu ENDP
```



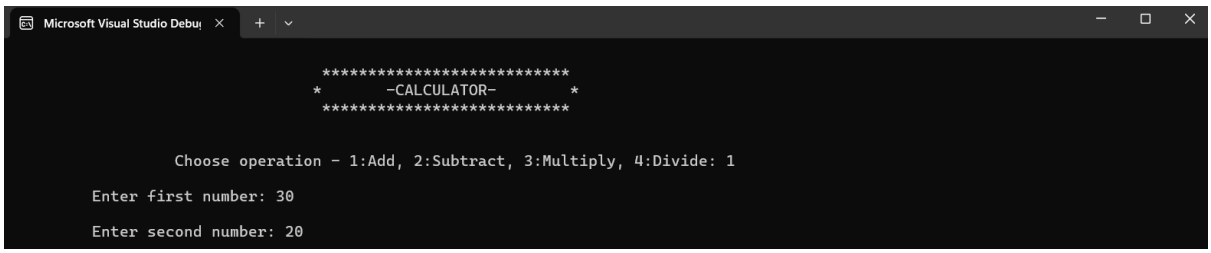
2. Get Number: Prompts the user to enter the first number. This procedure ensures that the input is correctly read and stored for subsequent calculations.

```
getNumber PROC
    call Crlf
    mov edx, OFFSET prompt1
    call WriteString
    call ReadInt
    ret
getNumber ENDP
```



3. **Get Second Number:** Prompts the user to enter the second number. Similar to the first input, this procedure ensures that the second number is correctly captured for the chosen arithmetic operation.

```
getSecondNumber PROC
    call Crlf
    mov edx, OFFSET prompt2
    call WriteString
    call ReadInt
    ret
getSecondNumber ENDP
```

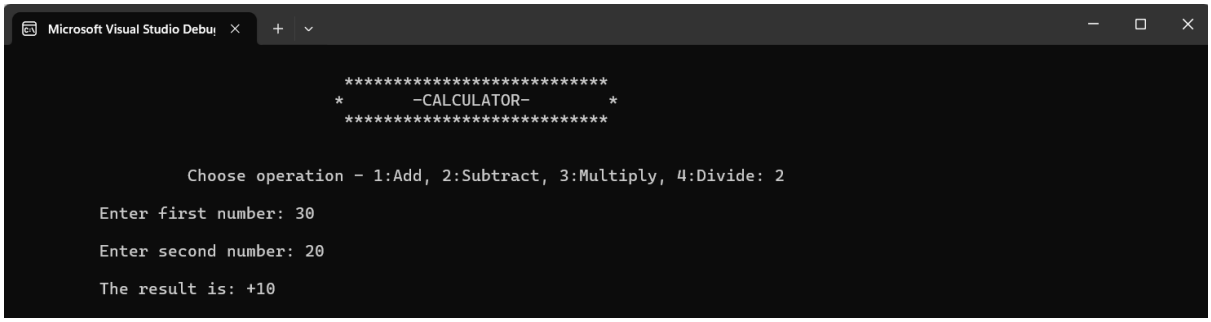


4. Addition: Adds two numbers and stores the result. This procedure implements the addition operation at the assembly level, showcasing the fundamental arithmetic capabilities of the CPU.

```
addition PROC
    add ecx, edx
    mov eax, ecx
    ret
addition ENDP
```

5. Subtraction: Subtracts the second number from the first and stores the result. This procedure highlights the use of assembly instructions for performing subtraction.

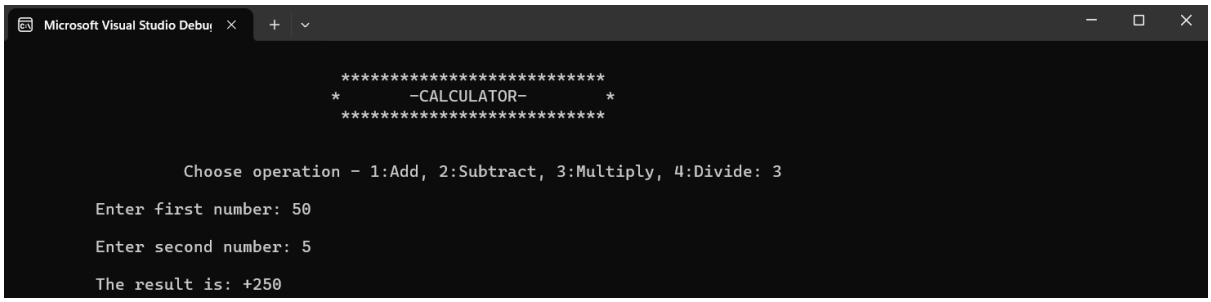
```
subtraction PROC
    sub ecx, edx
    mov eax, ecx
    ret
subtraction ENDP
```



6. Multiplication: Multiplies two numbers and stores the result. Multiplication in assembly involves specific instructions and handling of the result, which is often stored in multiple registers.

```
multiplication PROC
```

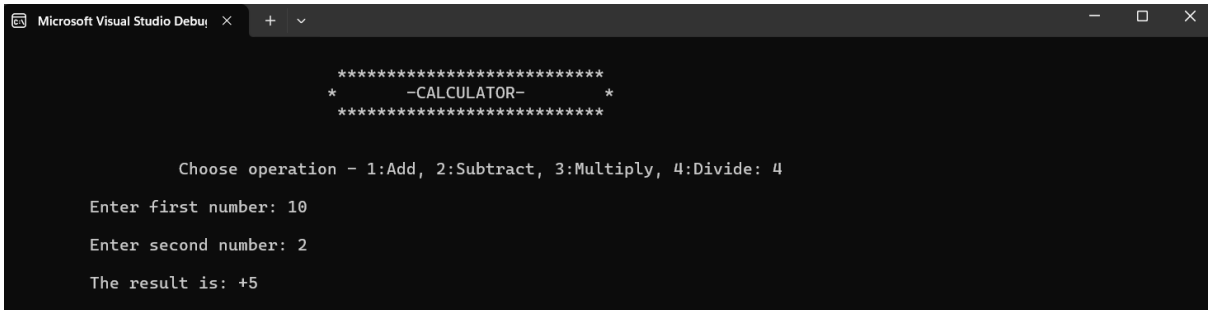
```
    mov eax, edx
    mul ecx
    ret
multiplication ENDP
```



7. Division: Divides the first number by the second, with a check for division by zero to handle errors appropriately. Division in assembly requires careful handling to manage both the quotient and the remainder, as well as error checking to prevent division by zero.

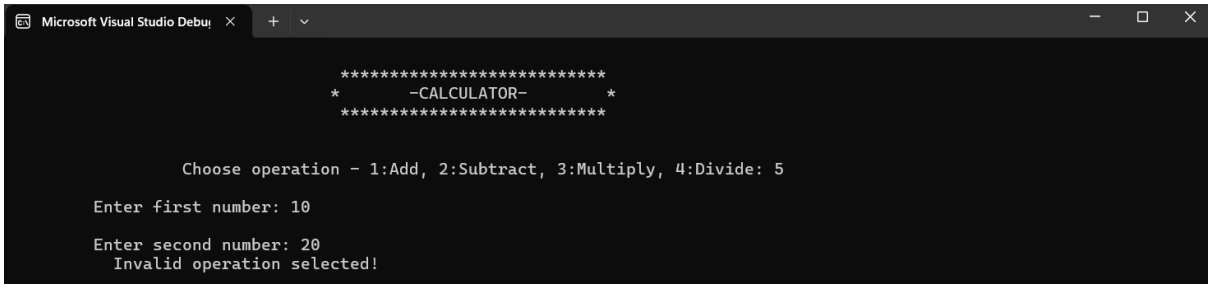
```
division PROC
    cmp edx, 0    ; Check if divisor is zero
    je divByZero  ; Jump to divByZero if it is zero
    mov edx, 0
    div ecx
    ret

divByZero:
    mov edx, OFFSET invalidOp
    call WriteString
    call Crlf
    ret
division ENDP
```



8. Invalid Operation: Handles cases where an invalid operation is selected by the user, displaying an error message. This procedure ensures robustness by gracefully handling invalid inputs.

```
invalidOperation PROC
    mov edx, OFFSET invalidOp
    call WriteString
    call Crlf
    ret
invalidOperation ENDP
```



Each procedure is carefully crafted to perform its specific task efficiently. The use of procedures promotes code reuse and makes the program easier to debug and extend.

Evaluation

The calculator was rigorously tested with various inputs to ensure that all operations performed correctly. Testing included:

- Performing each arithmetic operation (addition, subtraction, multiplication, and division) with different sets of numbers to verify correctness.

- Checking the handling of invalid operations by entering numbers outside the 1-4 range to ensure that the program responds appropriately.

- Ensuring that the division operation handles division by zero gracefully by displaying an error message.

The handling of division by zero was particularly crucial. The division procedure checks if the divisor is zero and jumps to a subroutine that displays an error message if true. This ensures that the program does not crash or produce undefined behavior, demonstrating the importance of robust error handling in assembly language programming.

The testing process highlighted the reliability and correctness of the implemented calculator. Each operation was verified to ensure accurate results, and the program's ability to handle errors gracefully was confirmed. This rigorous testing provided confidence in the functionality and robustness of the calculator.

Conclusions

This project provided valuable experience in assembly language programming and a deeper understanding of computer organization. By implementing a simple calculator, we enhanced our skills in handling user input, performing arithmetic operations, and managing program flow in assembly language. This project demonstrated the importance of understanding low-level programming and its impact on software performance and reliability.

The modular approach in the code ensured that each functionality was clearly defined, making the program easy to debug and extend. This experience underscored the value of careful planning and design in software development, particularly when working with low-level programming languages.

Through this project, we gained practical experience in assembly language programming, which will be beneficial for future projects involving low-

level programming or performance optimization. The skills acquired will also be useful in understanding and working with other programming languages and environments.

References

- Irvine, Kip R. "Assembly Language for x86 Processors." Pearson Education, 7th Edition.

- Microsoft Developer Network (MSDN) for documentation on MASM and Irvine32 library functions.

These references provided valuable guidance and information throughout the project. The Irvine textbook was particularly helpful in understanding the principles of assembly language programming, while the MSDN documentation provided essential details on the use of MASM and the Irvine32 library.

GitHub Link:

https://github.com/faisalameerCS/Calculator-in-Assembly-Language-x86/blob/main/Program%20Code